# ML EXPERIMENT-05

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
accuracy_score, confusion_matrix, classification_report
)
```

In [2]:
```
file_path = "car_evaluation.csv"
columns = [
"buying", "maint", "doors", "persons",
"lug_boot", "safety", "class"
]
df = pd.read_csv(file_path, header=None, names=columns)
print("Dataset Shape:", df.shape)
print("\nDataset Preview:")
print(df.head())
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Shape: (1728, 7)

Dataset Preview:
  buying  maint doors persons lug_boot safety  class
0  vhigh  vhigh     2       2    small    low  unacc
1  vhigh  vhigh     2       2    small    med  unacc
2  vhigh  vhigh     2       2    small   high  unacc
3  vhigh  vhigh     2       2      med    low  unacc
4  vhigh  vhigh     2       2      med    med  unacc

Missing Values:
buying      0
maint       0
doors       0
persons     0
lug_boot    0
safety      0
class       0
dtype: int64
```

In [3]:
```
df = df.drop_duplicates().reset_index(drop=True)

X = df.drop(columns=["class"])
y = df["class"]

X_encoded = pd.DataFrame(index=X.index)
encoders = {}
for col in X.columns:
    le = LabelEncoder()
```

```
    X_encoded[col] = le.fit_transform(X[col].astype(str))
    encoders[col] = le
target_encoder = LabelEncoder()
y_encoded = target_encoder.fit_transform(y.astype(str))
print("\nEncoded Features:")
print(X_encoded.head())
```

```
Encoded Features:
   buying  maint  doors  persons  lug_boot  safety
0       3      3      0        0         2       1
1       3      3      0        0         2       2
2       3      3      0        0         2       0
3       3      3      0        0         1       1
4       3      3      0        0         1       2
```

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
X_encoded, y_encoded,
test_size=0.25,
random_state=42,
stratify=y_encoded
)
```

In [5]:
```
rf = RandomForestClassifier(
n_estimators=100,
random_state=42
)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("\nAccuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(
classification_report(
y_test, y_pred,
target_names=target_encoder.classes_,
zero_division=0
)
)
```

```
Accuracy: 0.9768518518518519

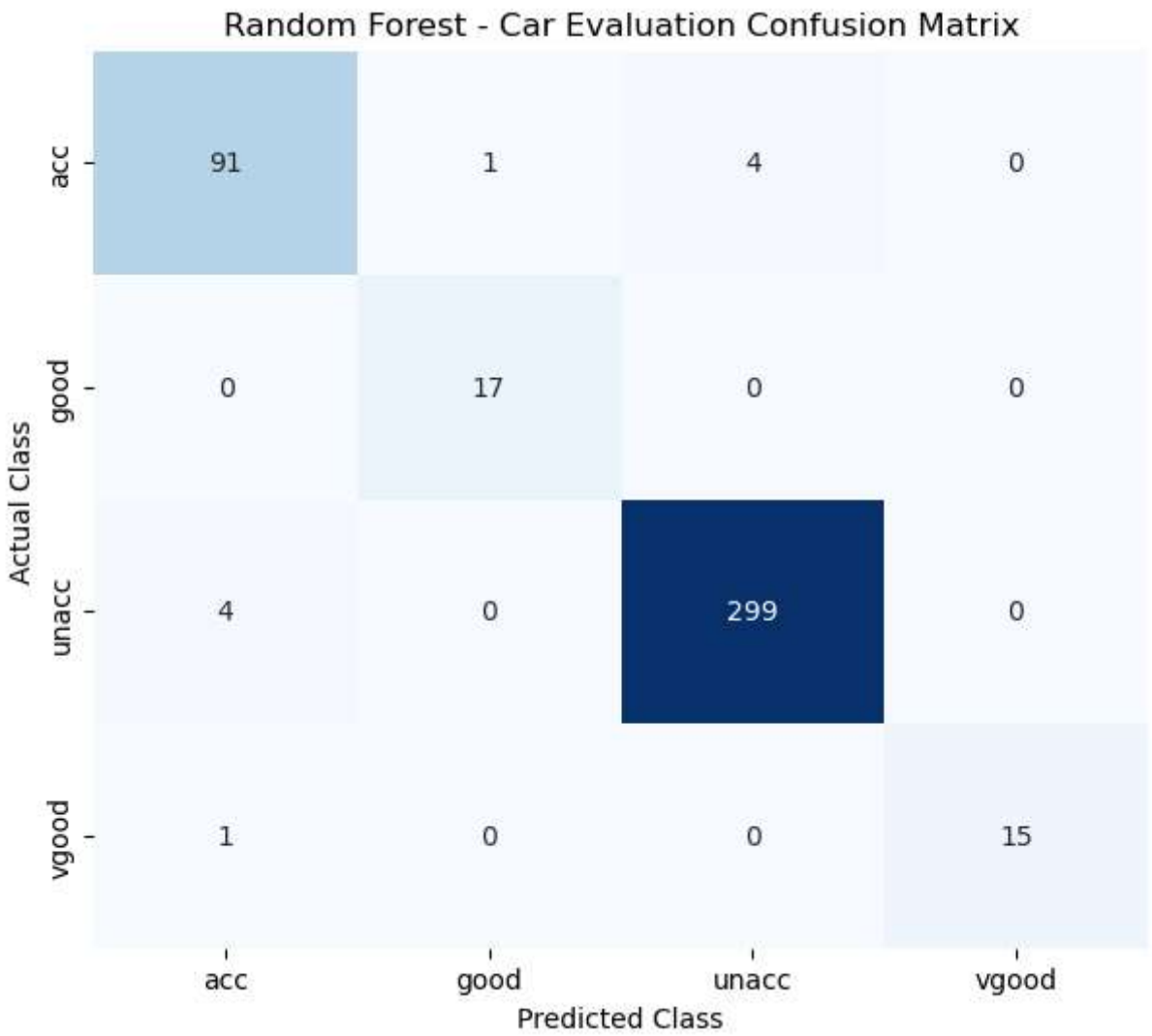
Confusion Matrix:
[[ 91   1   4   0]
 [  0  17   0   0]
 [  4   0 299   0]
 [  1   0   0  15]]

Classification Report:
              precision    recall  f1-score   support

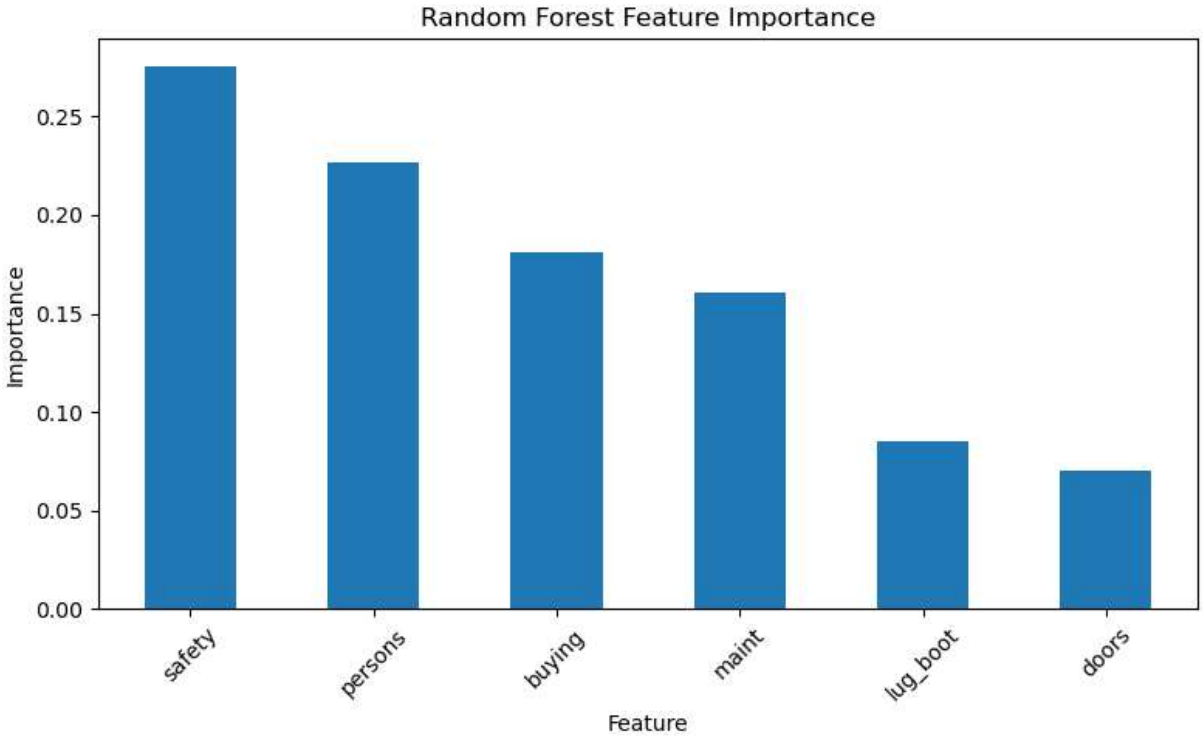
         acc       0.95      0.95      0.95        96
        good       0.94      1.00      0.97        17
       unacc       0.99      0.99      0.99       303
       vgood       1.00      0.94      0.97        16

    accuracy                           0.98       432
   macro avg       0.97      0.97      0.97       432
weighted avg       0.98      0.98      0.98       432
```

In [6]:
```
plt.figure(figsize=(7, 6))
sns.heatmap(
cm, annot=True, fmt="d", cmap="Blues", cbar=False,
xticklabels=target_encoder.classes_,
yticklabels=target_encoder.classes_
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Random Forest - Car Evaluation Confusion Matrix")
plt.show()
```




In [7]:
```
importance = pd.Series(
rf.feature_importances_,
index=X_encoded.columns
).sort_values(ascending=False)
plt.figure(figsize=(8, 5))
importance.plot(kind="bar")
plt.ylabel("Importance")
plt.xlabel("Feature")
plt.title("Random Forest Feature Importance")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```


Random Forest Feature Importance
Importance
0.25
0.20
0.15
0.10
0.05
0.00
safety
persons
buying
maint
lug_boot
doors
Feature